

Taller 2: Redes Neuronales

Ingeniería Industrial - Sistemas Avanzados de Producción

Entrega: 12 de Abril

1. Contexto y Motivación

En el marco de las ciudades inteligentes (*Smart Cities*) y la movilidad sostenible, los sistemas de bicicletas compartidas han dejado de ser solo un medio de transporte para convertirse en una red de sensores urbanos. El conjunto de datos **Capital Bikeshare** registra el flujo de alquiler de bicicletas en Washington D.C. entre los años 2011 y 2012, integrando información meteorológica y estacional.

Variable	Tipo	Descripción
instant	Entero	Índice de registro (identificador único).
dteday	Fecha	Fecha del registro (formato AAAA-MM-DD).
season	Categórica	Estación del año (1: invierno, 2: primavera, 3: verano, 4: otoño).
yr	Categórica	Año del registro (0: 2011, 1: 2012).
mnth	Categórica	Mes del año (1 a 12).
hr	Categórica	Hora del día (0 a 23).
holiday	Binaria	Indica si el día es festivo (1) o no (0).
weekday	Categórica	Día de la semana (0: domingo a 6: sábado).
workingday	Binaria	Si el día no es ni fin de semana ni festivo es 1, de lo contrario es 0.
weathersit	Categórica	Situación climática: 1: Despejado o nubes dispersas. 2: Niebla o nublado. 3: Lluvia ligera o nieve ligera. 4: Lluvia intensa, granizo o tormenta.
cnt	Entero	Variable Respuesta (Y): Cantidad total de bicicletas alquiladas (incluye usuarios casuales y registrados).

Cuadro 1: Diccionario de variables para el sistema de alquiler de bicicletas.

A diferencia de los modelos de demanda tradicionales, el alquiler de bicicletas no solo depende de factores temporales (hora del día, día de la semana), sino que es extremadamente sensible a variables exógenas como el clima y la sensación térmica. Para un Ingeniero Industrial, predecir con precisión esta demanda es un reto de optimización logística: un exceso de bicicletas en una estación genera costos de almacenamiento y subutilización, mientras que una escasez provoca la pérdida de clientes y fallos en el servicio de transporte multimodal.

El propósito de este taller es utilizar técnicas avanzadas —desde la regularización matricial hasta arquitecturas de redes neuronales— para capturar la complejidad de estos patrones y transformar datos crudos en una herramienta de decisión operativa.

2. Descripción de las Variables

Para el modelamiento de la demanda horaria y diaria, se han definido las siguientes variables contenidas en el dataset de la tabla 1.

3. Estructura de Evaluación

El taller se divide en tres fases fundamentales que cubren el ciclo de vida de un proyecto de analítica:

3.1. Fase 1: Desarrollo Analítico: Regularización Ridge.

En escenarios de alta dimensionalidad o cuando existe una fuerte correlación entre los predictores (como podría ocurrir entre la inversión en *TV* y *Radio*), el estimador de Mínimos Cuadrados Ordinarios (OLS) tiende a presentar una varianza excesiva. Para mitigar esto, se utiliza la **Regresión Ridge**, la cual añade una penalización a la magnitud de los coeficientes.

La función de costo a minimizar es la siguiente:

$$J(\beta) = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (1)$$

Tarea de Derivación Matemática:

Utilizando álgebra matricial, demuestre paso a paso cómo se llega a la solución cerrada del estimador Ridge. Para ello, siga esta ruta lógica:

1. **Representación Vectorial:** Expresé la función de costo $J(\beta)$ en términos de la matriz de diseño X , el vector de respuestas Y , el vector de parámetros β y el parámetro de regularización λ . *Pista: Recuerde que $\sum \beta_j^2$ se puede escribir como el producto interno del vector consigo mismo.*
2. **Expansión de la Forma Cuadrática:** Desarrolle el producto $(Y - X\beta)^T(Y - X\beta)$ y combine los términos semejantes con el término de penalización $\lambda\beta^T\beta$.
3. **Condición de Primer Orden:** Calcule el gradiente de la función de costo respecto al vector β (derivada matricial $\frac{\partial J(\beta)}{\partial \beta}$) e igualelo al vector nulo.
4. **Solución Cerrada:** Despeje el vector β para demostrar que el estimador Ridge está dado por:

$$\hat{\beta}_{Ridge} = (X^T X + \lambda I)^{-1} X^T Y$$

donde I es la matriz identidad de dimensiones adecuadas.

Preguntas Fase 1

- **Sobre la Invertibilidad:** ¿Por qué la suma de λI garantiza que la matriz resultante sea invertible incluso si $X^T X$ es singular (no invertible)?
- **Efecto del Parámetro λ :** ¿Qué sucede con el modelo y sus coeficientes cuando $\lambda \rightarrow 0$? ¿Y qué ocurre cuando $\lambda \rightarrow \infty$? Explique en términos de la complejidad del modelo (Sesgo vs. Varianza).

3.2. Fase 2: Optimización Numérica y Descenso de Gradiente

En esta fase, exploraremos el “motor” que permite entrenar modelos complejos como las Redes Neuronales: el algoritmo de **Descenso de Gradiente**. El objetivo es entender cómo la configuración de los hiperparámetros determina si un modelo logra converger al mínimo global de la función de pérdida o si falla en el intento. Para el desarrollo de esta sección, utilice el notebook disponible en el siguiente enlace:

GitHub: Semana 6 - Descenso de Gradiente.

Ubique el último bloque de código donde se configura la simulación:

```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
start_point = [0.0, 0.1]      # Punto de inicio
learning_rate = 0.0001       # Tasa de aprendizaje (eta)
num_iterations = 1000        # Número de pasos
```

Tareas de Experimentación

Utilizando la visualización de la superficie 3D y las curvas de nivel generadas por la malla (`np.meshgrid`), realice las siguientes variaciones y documente los cambios observados en la trayectoria del algoritmo:

1. Impacto de la Tasa de Aprendizaje (η):

- Mantenga el `start_point` y las `num_iterations` constantes.
- Incremente el `learning_rate` a un valor significativamente más alto (ej. 0.5 o 1.0). ¿Qué sucede con la trayectoria? ¿Logra llegar al centro de las curvas de nivel o comienza a oscilar/divergir?
- Reduzca el `learning_rate` a un valor extremadamente pequeño (ej. 10^{-7}). Observe el resultado tras 1000 iteraciones. ¿Es suficiente este tiempo de cómputo para alcanzar el mínimo?

2. Dependencia del Punto Inicial:

- Modifique el `start_point` a coordenadas alejadas del origen, por ejemplo `[6.0, -6.0]` o `[-5.0, 5.0]`.
- ¿El algoritmo llega siempre al mismo punto final? Explique si la topografía de la función $f(X, Y)$ sugiere la presencia de múltiples valles o mínimos locales.

Preguntas Fase 2

A partir de sus observaciones en la simulación, responda de manera crítica:

- **El fenómeno de “Overshooting”:** Explique físicamente qué ocurre cuando el paso ($\eta \cdot \nabla f$) es más grande que la distancia al mínimo. ¿Cómo se visualiza esto en el gráfico de curvas de nivel?
- **Eficiencia Computacional:** Si el algoritmo se detiene en la iteración 1000 pero aún se encuentra lejos del mínimo, ¿qué estrategia es mejor: aumentar el número de iteraciones o ajustar la tasa de aprendizaje? Justifique su respuesta.
- **Generalización:** En un problema de ingeniería con cientos de variables (como la optimización de una cadena de suministro), ¿por qué es peligroso confiar en un único punto de inicio aleatorio?

3.3. Fase 3: Redes Neuronales (MLP) y Ajuste de Hiperparámetros

En esta fase, implementaremos un **Perceptrón Multicapa (MLP)**, una arquitectura de red neuronal densa capaz de capturar relaciones no lineales complejas entre la inversión publicitaria y las ventas. A diferencia de la regresión lineal, aquí el modelo aprende representaciones abstractas en sus capas ocultas.

Para el desarrollo de esta sección, utilice el notebook disponible en:
GitHub: Semana 7 - Redes Neuronales y GridSearch.

Parte A: Evaluación de Métricas de Regresión

Ejecute el modelo `MLPRegressor` inicial con la configuración base (`hidden_layer_sizes=(16, 8, 4)`). Observe los resultados de las métricas **MAE**, **RMSE** y R^2 tanto para el conjunto de entrenamiento como para el de prueba (test).

Preguntas Fase 3A:

- **Diferenciación de Métricas:** Explique la diferencia técnica entre el Error Absoluto Medio (MAE) y la Raíz del Error Cuadrático Medio (RMSE). ¿Cuál de las dos penaliza con mayor severidad los errores grandes (*outliers*) en las ventas y por qué?
- **Diagnóstico de Desempeño:** Si el R^2 en el conjunto de entrenamiento es significativamente mayor que en el de prueba, ¿qué fenómeno está ocurriendo? Proponga una solución basada en lo aprendido en la Fase 1 del taller.
- **Interpretación de Residuos:** Al observar la gráfica de y_{real} vs y_{pred} , ¿se identifica algún patrón de error sistemático (ej. subestimación en ventas altas)?

3.3.1. Parte B: Optimización con GridSearchCV

La eficacia de una red neuronal depende críticamente de sus hiperparámetros. A continuación, se presenta una guía de referencia completa para orientar su proceso de optimización:

Guía de Referencia: Hiperparámetros del MLPRegressor

Tarea de Experimentación:

Utilizando la Tabla 2 como guía, modifique la parrilla de parámetros (`param_grid`) en el notebook y ejecute la búsqueda sistemática (*GridSearch*). Se sugiere variar estratégicamente los siguientes elementos:

1. **Arquitectura (`hidden_layer_sizes`):** Compare una red “profunda” (más capas, ej. $(32, 16, 8, 4)$) contra una “ancha” (una sola capa con muchas neuronas, ej. $(64,)$).
2. **Regularización (`alpha`):** Conecte esto con la Fase 1. Pruebe valores como $[1e-5, 1e-2, 1.0]$. ¿Cómo afecta un α muy elevado a la capacidad de aprendizaje?
3. **Tasa de Aprendizaje (`learning_rate_init`):** Conecte esto con la Fase 2. Analice cómo influye en el número de iteraciones necesarias para que el *Early Stopping* se active.

Nota sobre complejidad combinatoria: La grilla base tiene $4 \times 3 \times 3 = 36$ combinaciones $\times 3$ *folds* = 108 ajustes. Si añade `activation` y `batch_size`, supera fácilmente las 500 combinaciones.

Categoría	Parámetro	Descripción y Valores Sugeridos
Arquitectura	<code>hidden_layer_sizes</code>	Define profundidad y ancho de la red. Valores: [(16,8), (32,16), (64,32), (32,16,8), (64,32,16), (128,64,32)]. Más neuronas aumentan la capacidad pero también el riesgo de sobreajuste.
	<code>activation</code>	Función de activación por capa. “ <code>relu</code> ” (estándar y robusto), “ <code>tanh</code> ” (útil con datos centrados), “ <code>logistic</code> ”, “ <code>identity</code> ” (regresión lineal pura).
Optimizador	<code>solver</code>	Algoritmo de entrenamiento. “ <code>adam</code> ” (rápido y adaptativo), “ <code>sgd</code> ” (permite <i>momentum</i>), “ <code>lbfgs</code> ” (exacto pero lento; recomendado para datasets pequeños).
	<code>learning_rate_init</code>	Tasa de aprendizaje inicial. Valores: [1e-2, 5e-3, 1e-3, 5e-4, 1e-4]. Una tasa muy alta genera oscilaciones; una muy baja ralentiza la convergencia.
	<code>learning_rate</code>	Esquema de ajuste de LR (<code>sgd</code> únicamente). “ <code>constant</code> ”, “ <code>invscaling</code> ” o “ <code>adaptive</code> ” (reduce LR cuando el loss se estanca).
	<code>momentum</code>	Acumulación de gradiente en pasos previos (<code>sgd</code>). Valores: [0.0, 0.5, 0.9, 0.95]. El valor clásico en deep learning es 0.9.
	<code>batch_size</code>	Tamaño de mini-lote. Valores: [“ <code>auto</code> ”, 32, 64, 128, 256]. Lotes pequeños introducen más ruido pero suelen mejorar la generalización.
Regularización	<code>alpha</code>	Penalización L_2 sobre los pesos. Valores: [1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1.0]. Controla la complejidad del modelo (análogo a Ridge).
	<code>nesterovs_momentum</code>	Variante de momentum que anticipa el gradiente (<code>sgd</code>). Valores: [True, False]. Generalmente mejora la convergencia.
Early Stopping	<code>early_stopping</code>	Activa la parada anticipada. True/False. Detiene el entrenamiento si el error de validación deja de disminuir, previniendo sobreajuste.
	<code>validation_fraction</code>	Fracción de train usada internamente para monitorear la parada. Valores: [0.05, 0.10, 0.15, 0.20, 0.25].
	<code>n_iter_no_change</code>	“Paciencia”: épocas consecutivas sin mejora antes de parar. Valores: [5, 10, 20, 30, 50].
	<code>tol</code>	Mejora mínima para no contabilizar una época como “sin cambio”. Valores: [1e-5, 1e-4, 1e-3].
Inicialización	<code>random_state</code>	Semilla de reproducibilidad. Valores: [0, 42, 123]. Usar None introduce variabilidad y permite detectar inestabilidad.
	<code>max_iter</code>	Número máximo de épocas. Valores: [300, 500, 1000, 2000]. Con <code>early_stopping=True</code> raramente se alcanza el tope.

Cuadro 2: Catálogo de hiperparámetros del `MLPRegressor` de `scikit-learn` para tareas de regresión.

En ese caso, considere `RandomizedSearchCV` con `n_iter=50`.

Preguntas Fase 3B:

- **Consistencia de Parámetros:** Compare el valor óptimo de `alpha` encontrado por el Grid-Search con lo que analizó en la sección de Regresión Ridge. ¿Existe una relación entre la complejidad de la red y la necesidad de regularización?
- **Early Stopping:** Explique cómo este mecanismo ayuda a optimizar el tiempo de cómputo y a prevenir el sobreajuste basado en sus resultados.
- **Conclusión de Negocio:** Basándose en el mejor modelo encontrado, ¿cuál es el margen de error promedio que la empresa debería esperar al predecir sus ventas? ¿Es el modelo lo suficientemente robusto para decisiones presupuestales? Justifique.

3.4. Fase 4: Propagación hacia Adelante (Forward Propagation) Manual

En esta fase, el objetivo es comprender la mecánica interna de una Red Neuronal Multicapa. En lugar de predecir un valor continuo de ventas, clasificaremos el éxito de una campaña en tres categorías: **Bajo Impacto** (C_1), **Impacto Medio** (C_2) e **Impacto Alto** (C_3).

Arquitectura y Contexto

Considere una red neuronal con la siguiente estructura:

- **Capa de Entrada:** 3 características (x_1 : TV, x_2 : Radio, x_3 : Newspaper).
- **Capa Oculta (Capa 1):** 3 neuronas con función de activación **Sigmoide** (σ).
- **Capa de Salida (Capa 2):** 3 neuronas (una por clase) con función de activación **Softmax**.

Definición de Parámetros y Datos

Se proporcionan los siguientes pesos y sesgos (*biases*) ya entrenados para un mercado específico. El vector de entrada normalizado es $X = [0,5, 0,3, 0,2]^T$.

Capa 1 (Oculta):

$$W^{(1)} = \begin{pmatrix} 0,2 & 0,5 & -0,1 \\ 0,8 & -0,3 & 0,4 \\ -0,5 & 0,1 & 0,2 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0,1 \\ -0,2 \\ 0,0 \end{pmatrix}$$

Capa 2 (Salida):

$$W^{(2)} = \begin{pmatrix} 0,5 & -0,4 & 0,1 \\ 0,2 & 0,8 & -0,5 \\ -0,1 & 0,3 & 0,7 \end{pmatrix}, \quad b^{(2)} = \begin{pmatrix} -0,1 \\ 0,2 \\ 0,1 \end{pmatrix}$$

Tarea de Cálculo Manual

Realice los cálculos paso a paso para determinar a qué categoría pertenece la campaña publicitaria:

1. **Puntaje de la Capa Oculta ($Z^{(1)}$):** Calcule el producto punto $Z^{(1)} = W^{(1)}X + b^{(1)}$.

2. **Activación de la Capa Oculta ($A^{(1)}$):** Aplique la función sigmoide a cada elemento de $Z^{(1)}$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

3. **Puntaje de la Capa de Salida ($Z^{(2)}$):** Calcule $Z^{(2)} = W^{(2)}A^{(1)} + b^{(2)}$.

4. **Probabilidades de Clase ($A^{(2)}$):** Aplique la función **Softmax** al vector resultante $Z^{(2)}$ para obtener las probabilidades de cada clase (P_1, P_2, P_3):

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}}$$

Preguntas Fase 4

- **Interpretación de Resultados:** Según las probabilidades obtenidas, ¿en qué categoría clasifica la red a esta campaña publicitaria?
- **Función de Activación:** ¿Por qué utilizamos *Softmax* en la última capa en lugar de una función lineal o una sigmoide simple? Explique en términos de la suma de las probabilidades.
- **Rol del Bias:** ¿Qué sucedería con la capacidad de representación de la red si todos los vectores de sesgo (b) fueran cero?

3.5. Fase 5: Evaluación de Modelos y Toma de Decisiones (Caso: Mantenimiento Predictivo)

En la manufactura, una parada de máquina no planificada es un fallo crítico que genera cuellos de botella y sobrecostos. En esta fase, evaluaremos el desempeño de la Red Neuronal entrenada para el **Mantenimiento Predictivo (PdM)**, donde el objetivo es clasificar si una máquina fallará (Clase 1) o no (Clase 0).

Para el desarrollo de esta sección, utilice el notebook disponible en:
GitHub: Semana 8-9 — Clasificación y Evaluación de Modelos.

El Desafío del Desbalance de Clases

En plantas reales, las máquinas fallan rara vez (menos del 5 % del tiempo). Esto genera un *dataset* desbalanceado. Observe en el código del notebook cómo se calculó el diccionario `class_weights_dict` para manejar esta situación.

Preguntas Fase 5A:

- **Análisis de Pesos:** Si el modelo indica que los errores en la Clase 1 (Fallo) pesarán aproximadamente 14.7 veces más que en la Clase 0, ¿qué intenta corregir el algoritmo al configurar el parámetro `class_weight` durante el entrenamiento?
- **La Trampa de la Exactitud:** Explique por qué en este caso un *Accuracy* del 95 % podría considerarse un resultado deficiente para la gestión de mantenimiento si el modelo no logra detectar los fallos reales.

Interpretación de la Matriz de Confusión

Utilice la matriz de confusión generada por el *heatmap* de Seaborn en su notebook para completar el siguiente análisis. Suponga que para su planta de producción, el costo de una **reparación no planificada** (Fallo Real no detectado) es de USD 5,000 y el costo de una **inspección preventiva innecesaria** (Falsa Alarma) es de USD 200.

Tarea de Cálculo de Métricas: A partir de los valores obtenidos en su matriz de confusión (TN, FP, FN, TP), calcule manualmente:

1. **Precision (Precisión):** $\frac{TP}{TP+FP}$ — ¿Qué tan confiable es el modelo cuando emite una alerta de falla?
2. **Recall (Sensibilidad):** $\frac{TP}{TP+FN}$ — ¿Qué porcentaje de los fallos reales totales logró capturar el modelo?
3. **Costo Total de Error:** Calcule el impacto económico mensual del modelo utilizando la fórmula:

$$\text{Costo} = (FN \times 5000) + (FP \times 200)$$

Preguntas de Reflexión y Negocio Fase 5

- **Trade-off entre Recall y Precision:** En el contexto de evitar paradas de línea críticas, ¿preferiría usted un modelo con un *Recall* del 98 % pero una *Precision* del 60 %, o viceversa? Justifique su respuesta basándose en los costos calculados anteriormente.
- **Curva de Aprendizaje:** Observe la gráfica de *loss* (pérdida) y *accuracy* durante el entrenamiento. ¿Considera que 30 épocas son suficientes para que el modelo converja? ¿Qué sucede con el error de validación comparado con el de entrenamiento?
- **Conclusión Final del Taller:** Tras recorrer desde la regresión analítica hasta la clasificación con redes neuronales, ¿cómo puede el uso de métricas avanzadas (como el Recall) transformar la rentabilidad de una empresa de manufactura?

4. Instrucciones de Entrega y Rúbrica

4.1. Especificaciones del Entregable

El trabajo debe ser entregado en un único archivo en formato **PDF**, el cual debe ser cargado en el repositorio de GitHub asignado a cada grupo. El informe debe caracterizarse por ser **claro, profesional y conciso** (calidad sobre cantidad). No se busca un documento excesivo, sino un análisis con alta capacidad de síntesis que incluya:

- **Introducción:** Breve descripción del problema de la demanda de bicicletas y los objetivos de optimización y clasificación.
- **Desarrollo Técnico:** Evidencia organizada de las 5 fases. Se deben incluir los cálculos manuales (Fases 1 y 4) y las visualizaciones clave de los notebooks (Fases 2, 3 y 5).
- **Análisis Crítico e Interpretación:** Es el componente más importante. Cada resultado numérico o gráfico debe ser interpretado en el contexto del negocio (logística de transporte o disponibilidad de planta). No se aceptará código sin una explicación de su impacto en la toma de decisiones.

- **Conclusiones:** Reflexión sobre la arquitectura de red óptima y la importancia de las métricas de sensibilidad en entornos industriales.
- **Referencias:** Listado de fuentes consultadas (documentación de *scikit-learn*, libros de texto o artículos técnicos) siguiendo normas APA o IEEE.

4.2. Rúbrica de Evaluación

La calificación se basa en la capacidad del estudiante para conectar la fundamentación matemática con la ejecución computacional y la realidad operativa:

Criterio	Peso	Descripción
Rigor Matemático	25 %	Derivación correcta de la Regresión Ridge (Fase 1) y precisión en el cálculo manual de <i>Forward Propagation</i> (Fase 4).
Optimización y Modelado	25 %	Experimentación lógica con el Descenso de Gradiente (Fase 2) y búsqueda sistemática de hiperparámetros (<i>GridSearch</i>) en la Fase 3.
Análisis de Impacto	40 %	Interpretación de métricas de clasificación (Recall vs. Precision) y cálculo del impacto económico en el caso de mantenimiento (Fase 5). Se evalúa la profundidad del “Análisis Crítico”.
Comunicación y Fuentes	10 %	Informe claro, redacción sin excesos, uso correcto de bibliografía y organización del repositorio en GitHub.

Nota Importante: Los resultados numéricos correctos que carezcan de una interpretación analítica o una conclusión de negocio asociada se penalizarán con el 50 % del valor del punto correspondiente.